A Comparative Analysis of Apache HBase and MySQL: Architecture, Performance, and Industrial Adoption in Real-Time Messaging

Farra Nurzahin binti Zaharil
Anuar
Faculty of Computing
Universiti Teknologi Malaysia
Johor Bahru, Malaysia
farranurzahin@graduate.utm.my

Abstract— **The rapid growth of global digital data has made traditional Relational Database Management Systems (RDBMS) like MySQL increasingly difficult to scale for massive datasets requiring random read/write access. This paper evaluates Apache HBase, a column-oriented NoSQL database inspired by Google's Bigtable, as a solution for real-time processing of billions of records. The HBase master-slave architecture and its multidimensional data model is examined, which allow for horizontal scalability and high throughput on top of HDFS. Through a comparative analysis, it demonstrates that while MySQL remains more efficient for read-intensive applications and structured data, HBase provides significant advantages for write-heavy workloads and petabyte-scale operations. A case study of Facebook Messenger validates these findings, highlighting the necessity of HBase for maintaining consistency and availability at massive industrial scales.**

Keywords—Big Data, Apache HBase, NoSQL, Hadoop, HDFS, MySQL, Distributed Databases, Facebook Messenger

I. INTRODUCTION

Globally, digital data has grown at an unprecedented rate. According to some estimates, 90% of the world's data was produced in the previous years and is expected to grow by 40% every year [1]. This deluge of digital information is referred to as "big data". However, using traditional Relational Database Management Systems (RDBMS) to measure or scale up with massive amounts of data and random read/write access will be challenging [2].

Reference [3] states the emergence of NoSQL databases which loosened strict ACID transaction constraints in favor of high availability and performance, solved many of these drawbacks. As part of Hadoop ecosystem, HBase represents a specialized type of columnar NoSQL database that draws inspiration from Google's BigTable, Apache Hive, a data warehouse built on top of Hadoop and Apache ZooKeeper, a distributed system coordination tool [3]. It also provides a distributed, fault-tolerant and highly scalable system built on top of the Hadoop Distributed File System (HDFS). Moreover, HBase provides real-time, random access capabilities for massive dataset by organizing data into multidimensional sparse maps [4]. HBase has seen widespread industry adoption by organizations since it introduced as a Hadoop sub-project such as Facebook, Yahoo! and Adobe as it is capable of managing numerous read/write requests across tables containing billions of rows [2].

Before the adoption of HBase, organizations employed sharded MySQL systems but these topologies were inconsistent and operationally challenging when scalling to petabyte-level quantities [3]. The objectives of this paper are to: (1) describe Apache HBase's architecture and data model; (2) analyze its major performance features against MySQL; (3) offer a systematic comparative evaluation against MySQL and (4) discuss a real-world case.

II. APACHE HBASE ARCHITECTURE AND DATA MODEL

A. Apache HBase

Apache HBase, an open-source column-oriented NoSQL database, inspired by Google's Bigtable that can query datasets containing billions of records in real time [5]. It uses Apache ZooKeeper for cluster coordination, failure detection and configuration management and HDFS for data persistence as part of the Apache Hadoop ecosystem. The architecture is governed by a Master Node that oversees metadata and balances data regions across multiple RegionServers, which serve as worker nodes responsible for managing sorted data shards and processing client requests. This finding is supported by [4] where HBase uses a master/slave design in which HMaster (master) is in charge of allocating regions to HRegionServers (slaves) and recovering from its failures while HRegionServers is in charge of handling reading and writing requests from clients. This architecture can be viewed as labelled in Fig. 1. Moreover, high-speed, random access to distributed data is made possible by HBase which uses the MapReduce programming model to carry out SQL-like searches and aggregations while guaranteeing fault tolerance through a leader election procedure for its master instances [4].

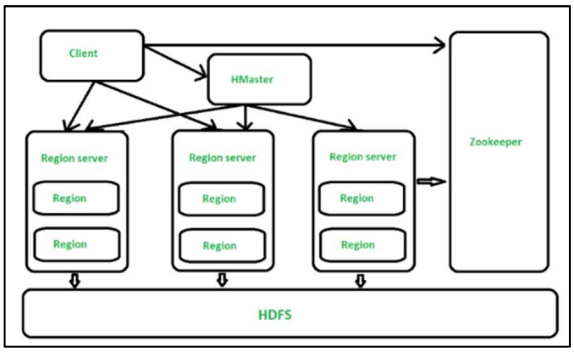

Fig. 1. Apache HBase Architecture

B. *HBase Data Model*

The Apache HBase data model is a distributed, non-relational Column Family store that arranges data into labeled tables using cells as the fundamental storage unit [6]. Multiple versions of a given data item can be saved in the same location since each cell is uniquely recognized by a combination of its row ID, column-family name, column name and version identifier. Reference [6] states the HBase model can be abstracted into a three-dimensional space by using the version dimension to store sequential data, like time-series timestamps or snapshot identifiers, whereas traditional relational models are seen in two dimensions (rows and columns). Expanding on this multidimensional viewpoint, [2] further describes the architecture as a four-dimensional model as shown in Fig. 2 by differentiating the precise coordinates needed to find a cell: Row Key, Column Family, Column Qualifies and Timestamp. This multidimensional map ensures that table rows are ordered according to their primary row key for faster query results and higher throughput which allow large datasets to be stored across the HDFS [2].

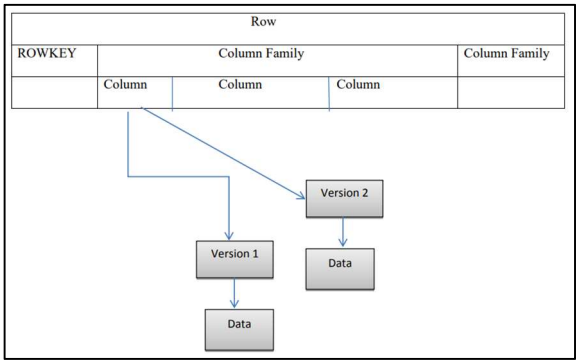

Fig. 2. HBase Data Model

III. PERFORMANCE CHARACTERISTIC

HBase's columnar storage model and distributed writing architecture define its performance profile. The Yahoo! Cloud Serving Benchmark (YCSB) methodology has been frequently used in benchmarking studies to identify the following features.

For write-intensive workloads, HBase demonstrates significant advantages. In a comparative study, [7] discovered that HBase consistently performed better than MySQL in terms of update delay and data loading runtime across datasets with between 100000 and one million records as shown in Fig. 3. This advantage is attributed to how HBase performs better in update operations that MySQL because it records transactions using log files and cache memories which significantly reduces input/output (I/O) operations.

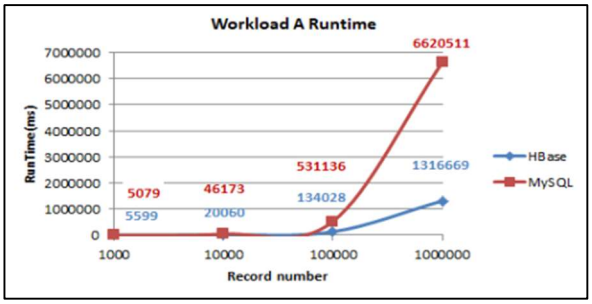

Fig. 3. HBase vs. MySQL Runtime

For read operations, experimental data in Fig. 4 shows that MySQL maintains faster runtimes and reduced read latency workloads than HBase [7]. HBase's read performance is hindered by its need to compare all data copies to ensure the most recent version is returned, a process that introduces significant overhead. Even if both systems' read latency is greatly impacted by increasing the number of operations, MySQL remains more efficient for read-intensive applications.

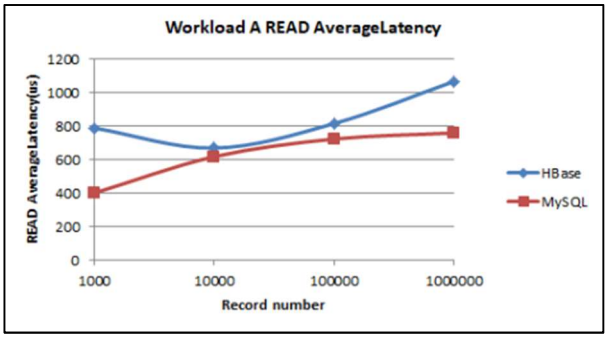

Fig. 4. HBase vs. MySQL read latency

For scalability, HBase demonstrates near-linear throughput scaling as cluster nodes are added, a characteristic MySQL sharded architectures cannot match without significant application-level engineering. For IoT data streams, [8] showed that Redis-HBase hybrid time-series model achieved intake and query efficiency significantly better than standalone HBase which prove that complementing caching layers can further improve HBase's baseline performance.

IV. Real Word Case Study: Facebook Messenger

One of the most widely cited industrial deployments of Apache HBase is Facebook's adoption for its Messenger platform, documented by [3]. Prior to HBase, Facebook's messaging infrastructure operated on a sharded MySQL architecture. While functionally adequate initially, this approach required manual shard rebalancing and struggled with the consistency and availability requirements of a messaging system serving hundreds of millions of concurrent users. After evaluating alternatives like Apache Cassandra and Project Voldemort, Facebook selected HBase specifically for its strong per-row consistency and high write throughput required to sustain billions of messages per day. The selection criteria prioritised three properties: strong per-row consistency for message ordering, high write throughput to sustain billions of messages per day and proven operability at massive scale.

Reference 3 states the deployment required critical engineering enhancements to the Hadoop ecosystem to meet real-time performance demands as mention. Facebook developed the "AvatarNode" to provide a high-availability master for HDFS, eliminating single points of failure, and integrated Bloom filters to optimize read efficiency. These engineering contributions were subsequently returned to the open-source Apache community. This case study validates that HBase's architecture sustains write throughput at petabyte scale when properly configured, confirming the theoretical performance characteristics in Section III.

V. Comparative Analysis: HBase vs. MySQL

The decision between HBase and MySQL is mainly based on the particular demands of the workload, weighing the trade-offs between distributed scalability and relational consistency. HBase performs well in settings with large data volumes and frequent write operations but MySQL is still the best option for structured data needing extensive joins and low-latency, single-record lookups inside smaller datasets [7]. The following Table 1 provides a detailed comparison of these two systems according to [1], [7] and [9].

TABLE 1
Comparative Analysis: HBase vs. MySQL

Criteria	MySQL (Relational RDBMS)	Apache HBase (Columnar HBase)
Data Model	Fixed Schema; predefined rows and columns	Flexible schema: sparse column families with versioned cells
Storage Approach	Row-oriented storage (full row read per query)	Column-family oriented; only queried columns are read.
Scalability	Vertical scaling (scale-up); limited nodes	Horizontal scale-out across commodity nodes; linear scalling
Query Language	Full SQL: JOIN, GROUP BY, subquries	REST-based APIs; to interact with databases
Write Speed	Moderate	High throughput
Read Latency	Low for indexed primary-key lookups	Low for row-key lookups; Bloom filters reduce false reads
Best Use Case	OLTP: e-commerce, ERP, CRM, banking transactions	Big Data; IoT, time-series, social feeds, event logging

The analysis reveals that the systems serve complementary use cases. MySQL's strength lies in its mature SQL interface, full ACID compliance, and join capabilities, which remain essential for OLTP workloads such as financial transactions, e-commerce order management and ERP systems. HBase excels in workloads characterised by high write rates, sparse schemas, and predictable row-key access patterns. The absence of native multi-row transactions in HBase restricts its applicability to workloads where cross-row atomicity is not required [10].

VI. Conclusion

This paper has provided a comprehensive evaluation of Apache HBase as a critical component of the modern big data ecosystem. The investigation into its architecture reveals that its reliance on HDFS and ZooKeeper provides a fault-tolerant and highly scalable foundation capable of managing billions of rows. Comparative performance testing demonstrates that HBase's distributed writing architecture significantly reduces I/O operations for write-intensive applications, although at the cost of higher read latency compared to MySQL due to its version-comparison overhead. The real-world implementation at Facebook highlights how HBase addresses the operational challenges of sharded MySQL systems by providing strong consistency and simplified shard management at massive scale. While HBase is ideally suited for IoT data streams and social feeds, its efficiency can be further improved through hybrid models like Redis-HBase to optimize query intake. Ultimately, HBase remains a vital tool for organizations prioritizing horizontal scalability and write performance in their data storage strategies.

Acknowledgment

REFERENCES

[1] Mershad, K. (2019, February). MQL: mixed query language for querying mySQL and HBase databases. In *2019 International Conference on Innovative Trends in Computer Engineering (ITCE)* (pp. 124-129). IEEE.

[2] Nalla, R. R., Sengupta, S., Novillo, J., & Rezk, M. (2015). A case study on Apache HBase.

[3] Borthakur, D., Gray, J., Sarma, J. S., Muthukkaruppan, K., Spiegelberg, N., Kuang, H., ... & Aiyer, A. (2011, June). Apache hadoop goes realtime at facebook. In *Proceedings of the 2011 ACM SIGMOD International Conference on Management of data* (pp. 1071-1080).

[4] Vora, M. N. (2011, December). Hadoop-HBase for large-scale data. In *Proceedings of 2011 International Conference on Computer Science and Network Technology* (Vol. 1, pp. 601-605). IEEE.

[5] Razavi, S. M., Kahani, M., & Paydar, S. (2021). Big data fuzzy C-means algorithm based on bee colony optimization using an Apache Hbase. *Journal of big data*, *8*(1), 64.

[6] Han, D., & Stroulia, E. (2012, September). A three-dimensional data model in hbase for large time-series dataset analysis. In *2012 IEEE 6th International Workshop on the Maintenance and Evolution of Service-Oriented and Cloud-Based Systems (MESOCA)* (pp. 47-56). IEEE.

[7] Bousalem, Z., Guabassi, I. E., & Cherti, I. (2019). Relational databases versus HBase: an experimental evaluation. *Adv. Sci. Technol. Eng. Syst. J*, *4*(2), 395-401.

[8] Ma, R., Zhou, W., & Ma, Z. (2024). An efficient nosql-based storage schema for large-scale time series data. *Journal of Database Management (JDM)*, *35*(1), 1-21.

[9] Tudorica, B. G., & Bucur, C. (2011, June). A comparison between several NoSQL databases with comments and notes. In *2011 RoEduNet international conference 10th edition: Networking in education and research* (pp. 1-5). IEEE.

[10] Tan, H., Meehan, J., Dou, W., & Madden, S. (2022). The time machine in columnar NoSQL databases: The case of Apache HBase. Future Internet, 14(3), 92. https://doi.org/10.3390/fi14030092

Logbook

Name: Farra Nurzahin binti Zaharil Anuar
Student ID: A23CS0079

Date	Search Keywords / Source Used	AI Prompt Used	Task Description
15/4/2026	Apache HBase Scopus	None	Search Scopus and Google Scholar to confirm sufficient resources exists. Found 11 papers and read its abstracts. Start to write introduction.
16/4/2026	Hbase architecture diagram (IEEE Xplore)	None	Found an architecture from the papers and cited it in my paper.
16/4/2026	Hbase data model (Google Scholar)	Asked Claude to explain about HBase data model stated by the paper.	Validate the points by reviewing the paper and take note for the Data Model part of my paper.
16/4/2026	Hbase data model (IEEE Xplore)	Asked Claude to find more reliable sources about data model that is accessible.	Write the data model to support earlier points.
19/4/2026	Hbase performance (Scopus)	None	Read the paper that tested the HBase against MySQL and wrote down the key performances.
11/5/2026	Hbase performance (Scopus)	None	Found about Redis-Hbase mix system and include in the performance part.
16/5/2026	Hbase case study (IEEE Xplorer)	None	Read a paper about how Facebook uses HBase. Start to write the real-world case study part.
17/5/2026	Hbase MySQL (Scopus)	Asked Claude for help to build a comparison table for HBase and MySQL.	Proof read the papers and take note of the points to be written in my report.
17/5/2026	Hbase MySQL advantages (IEEE Xplorer)	None	Find more on the comparison between HBase and MySQL.
18/5/2026	None	None	Write conclusion and abstract based on the research, listed out all of the references.
21/6/2026	None	None	Review the full paper before submission.